

| CHAPTER 4 |

DATABASE CONSTRAINTS AND NORMALISATION

What is a database?

Database can be defined as, “an organized collection of information stored electronically that lets multiple people safely access, manage, and update the same data without conflicts or duplicates.”

Database constraints are like automatic referees that check if data follows the rules and block anything that doesn't, to keep your information clean, consistent, and trustworthy. They avoid:

- *Data Corruption* – Garbage values entering the system
- *Inconsistency* – Contradictory information in the database
- *Integrity Violations* – Broken relationships between data
- *Business Rule Breaches* – Actions that don't make real-world sense
- *Unreliable Information* – Data you can't trust for decisions

What is a Database Constraint?

con·straint

[kən-'strānt]

**the state of being checked, restricted, or
compelled to avoid or perform some action**

In database, constraints are sets of rules that check data values against predefined conditions and restrict database operations that don't align with those rules, to avoid data corruption, inconsistency, and business logic violations.

DATABASE CONSTRAINTS

In database, we can categorize constraints as:

1. Domain Constraints
2. Integrity Constraints
3. Key Constraints
4. Referential Constraints

DATABASE CONSTRAINTS

- Domain Constraint

These define the legal values for an attribute.

It ensures that the data being entered is of the correct "type" and follows specific business rules.

Enforced by: Data types (e.g., `INT`, `VARCHAR`), `CHECK` constraints, and `NOT NULL`

DATABASE CONSTRAINTS

- Entity Integrity Constraints

This rule states that every table must have a Primary Key and that this key cannot be NULL.

It ensures that every single row (entity) in your table can be uniquely identified.

Enforced by: **PRIMARY KEY**

DATABASE CONSTRAINTS

- Referential Integrity Constraints

This maintains the relationship between tables.

It states that a Foreign Key must either match a value in the parent table's primary key or be NULL.

This prevents "orphaned" records.

Enforced by: **FOREIGN KEY**

DATABASE CONSTRAINTS

- Key Constraints

While similar to Entity Integrity, this category ensures that any attribute (or set of attributes) intended to be a key (Primary, Candidate, or Unique) must have unique values across the table.

Enforced by: **UNIQUE** and **PRIMARY KEY**.

Example: Even if Email isn't the primary key, a unique key constraint ensures no two users sign up with the same email.

Constraint Type	Level	Main Purpose	SQL Keywords	Key Rule to Remember
Domain	Column	Restricts the type and range of data in a cell.	INT, VARCHAR, CHECK, NOT NULL, DEFAULT	Filters the "legal" values for a specific attribute.
Entity Integrity	Table	Ensures every row can be uniquely identified.	PRIMARY KEY	The primary key cannot be NULL and must be unique.
Referential Integrity	Table	Maintains consistency between linked tables.	FOREIGN KEY, REFERENCES	A foreign key must match an existing primary key or be NULL.
Key Constraints	Table	Ensures uniqueness of data across rows.	UNIQUE, PRIMARY KEY	Prevents duplicate data; UNIQUE allows NULLs, PRIMARY KEY does not.

While a standard **CHECK** constraint is tied to a single row or table, an Assertion is a standalone database object that ensures a specific condition is true across the entire database, often involving multiple tables simultaneously.

ASSERTIONS

An assertion is a declarative way to express business rules that are too complex for simple column or table constraints.

- *Scope*: Database-wide (can reference any table).
- *Activation*: The DBMS checks the assertion whenever any table mentioned in its condition is modified (Insert, Update, or Delete).
- *Result*: If the condition becomes FALSE, the entire transaction that caused the change is rolled back and error message is displayed.

ASSERTION Example

```
CREATE TABLE Products (  
    ProductID INT PRIMARY KEY,  
    ProductName VARCHAR(50),  
    StockQuantity INT,  
    CONSTRAINT chk_stock_positive CHECK  
    (StockQuantity >= 0));
```

Condition	SQL Query (Action)	Result	Logic
True (Pass)	INSERT INTO Products VALUES (1, 'Laptop', 10);	Success	10 >= 0 is True. The row is added.
False (Fail)	INSERT INTO Products VALUES (2, 'Mouse', -5);	Error	-5 >= 0 is False. The DB rejects the entry.

Decomposition in SQL

In SQL and database design, decomposition is the process of breaking down a large, complex table into several smaller, more manageable tables. This is a fundamental part of Normalization.

Student_ID	Student_Name	Course_ID	Course_Name	Instructor	Dept_Name	Dept_Head
101	Alice	C1	Algebra	Prof. Ray	Math	Dr. Gauss
101	Alice	C2	Physics	Prof. Gale	Science	Dr. Curie
102	Bob	C1	Algebra	Prof. Ray	Math	Dr. Gauss
103	Charlie	C3	History	Prof. Lee	Humanities	Dr. Plato

The primary goal is to eliminate redundancy (saving space) and prevent anomalies (errors during inserting, updating, or deleting data).

- Insertion Anomalies: You can't add certain data because other data is missing (e.g., you can't add a new course until a student enrolls in it).
- Update Anomalies: If a teacher changes their name, you have to update every single row where that teacher appears.
- Deletion Anomalies: Deleting a student might accidentally delete the only record of a specific course existing.

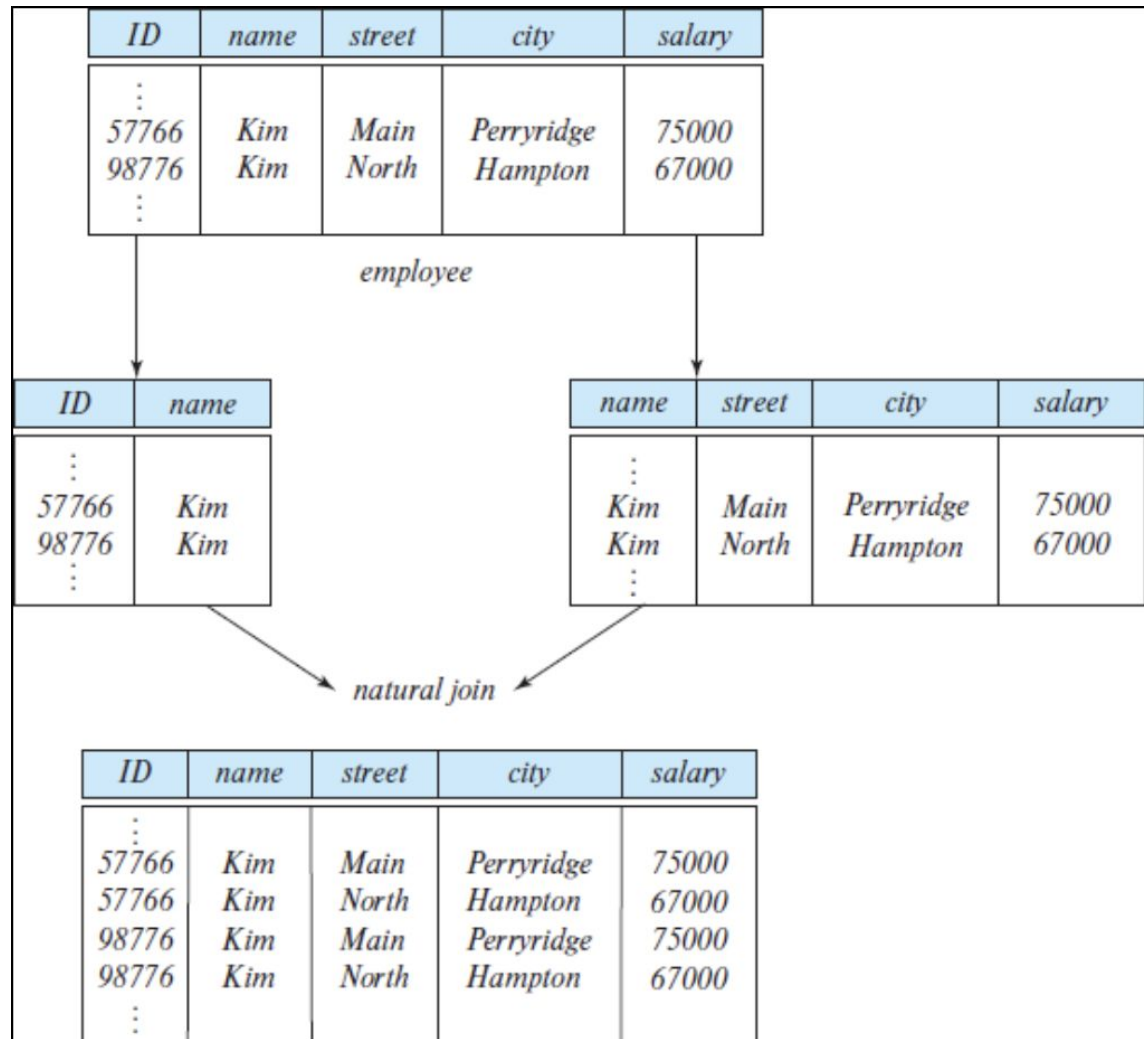
Bad decomposition

If a relation X is decomposed into relations

A and B,

Then the decomposition is said to be bad decomposition if

$$A \cup B \neq X$$



Normalization

Normalization is the systematic process of organizing data in a database to reduce redundancy (duplication) and improve data integrity.

In simpler terms, it involves decomposing a large, "flat" table into smaller, related tables so that each piece of information is stored in exactly one place.

Normalisation vs Decomposition

Normalization is the "End State" we want to reach. Decomposition is the "Action" we take to get there.

Feature	Normalization	Decomposition
Nature	A methodology or goal.	A mechanism or action.
Purpose	To eliminate redundancy and anomalies.	To physically split a table into parts.
Scope	Includes identifying keys and dependencies.	Focuses on how to split without losing data.
Rules	Follows Normal Forms (1NF, 2NF, 3NF).	Follows properties (Lossless Join, Dependency Preservation).

First Normal Form

A relation is said to be in First Normal Form if

- Each column contains only atomic (indivisible) values with no multiple values, lists, or arrays in a single cell
- All entries in a column are of the same data type
- Each column has a unique name
- There are no repeating groups or columns (such as Phone1, Phone2, Phone3)
- The order of rows and columns is insignificant to data integrity
- The table has a primary key that uniquely identifies each row

Stage 1 (Unnormalized)

Dish	Ingredients
Omlette	Egg (2 pc), Oil (1 tbsp)
Sandwich	Bread (2 pc), Cheese (1 pc)
Toast	Bread (2 pc)
Latte	Espresso (1 tbsp), Steamed milk (0.5 cup)

This is a human-readable list where multiple data points (ingredients and quantities) are grouped into a single cell. It is difficult for a computer to process.

Stage 2 (Flat File)

Dish	Ingredient 1	Ingredient 2	Ingredient 3	Ingredient 4	Ingredient 5
Omlette	Egg (2 pc)	Oil (1 tbsp)			
Sandwich	Bread (2 pc)	Cheese (1 pc)			
Toast	Bread (2 pc)				
Latte	Espresso (1 tbsp)	Steamed milk (0.5 cup)			
Club Sandwich	Bread (3 pc)	Turkey (2 slice)	Bacon (2 strip)	Lettuce (1 leaf)	Mayo (1 tbsp)

Data separated into columns (Ingredient 1, 2, 3)

Stage 3 (1NF)

Dish	Ingredient	Quantity
Omlette	Egg	2 pc
Omlette	Oil	1 tbsp
Sandwich	Bread	2 pc
Sandwich	Cheese	1 pc
Toast	Bread	2 pc
Latte	Espresso	1 tbsp
Latte	Steamed milk	0.5 cup

Stage 4 (Refined 1NF - Atomically Scalable Data)

Dish	Ingredient	Amount	Unit
Omlette	Egg	2	pc
Omlette	Oil	1	tbsp
Sandwich	Bread	2	pc
Sandwich	Cheese	1	pc
Toast	Bread	2	pc
Latte	Espresso	1	tbsp
Latte	Steamed milk	0.5	cup

Significance of Normalization

1. *Eliminating Data Redundancy*

Normalization removes duplicate data by ensuring that each piece of information is stored in only one place. This creates a single source of truth that is easier to maintain and update consistently across the entire database.

2. *Ensuring Data Integrity*

Through the enforcement of dependencies and relationships, normalization maintains data accuracy and consistency. When data exists in only one location, contradictory information cannot arise, and update, insertion, and deletion anomalies are prevented.

Significance of Normalization....

3. *Improving Query Performance*

Normalization creates smaller, more focused tables that can improve query performance as queries accessing specific data subsets can retrieve information faster.

4. *Facilitating Database Maintenance*

Normalized databases are easier to modify and extend. Adding new attributes or relationships becomes straightforward,

Question: Convert the following into 1NF

StudentID	StudentName	ContactInfo	Courses_Taken
101	Alice Smith	alice@email.com, 555-0101	CS101, MATH202
102	Bob Jones	bob@email.com	CS101
103	Charlie Day	charlie@email.com, 555-0103	CS101, PHYS101, ART110

Answer:

StudentID	StudentName	ContactEmail	ContactPhone	Course
101	Alice Smith	alice@email.com	555-0101	CS101
101	Alice Smith	alice@email.com	555-0101	MATH202
102	Bob Jones	bob@email.com	NULL	CS101
103	Charlie Day	charlie@email.com	555-0103	CS101
103	Charlie Day	charlie@email.com	555-0103	PHYS101
103	Charlie Day	charlie@email.com	555-0103	ART110

Second Normal Form

A table is in 2NF if:

1. It is already in 1NF (no repeating groups, atomic values).
2. No non-key attribute is partially dependent on any composite candidate key.

To reach Second Normal Form (2NF), a table must first satisfy all the requirements of 1NF, and then go one step further to fix "partial" *dependencies*.

DEPENDENCIES

A dependency is a relationship between attributes (columns). It occurs when the value of one attribute uniquely determines the value of another.

Think of it as a mathematical function: $X \rightarrow Y$ (read as "X determines Y"). If you know the value of X, you can look up and find exactly one specific value for Y.

DEPENDENCIES

A functional dependency $X \rightarrow Y$ exists if for every instance of X, there is exactly one value of Y.

- Determinant: The left-hand side X. It "determines" the other value.
- Dependent: The right-hand side (Y). Its value "depends" on X.
- The Rule: If two rows have the same X, they must have the same Y.

Example:

1. Functional Dependency

This is the most common type. It means that for every unique value in Column A, there is exactly one value in Column B.

Example: Roll_no $\rightarrow$ Email.

If I know your Roll no is ACE080BCT099, there is only one possible email associated with it.

2. Fully Functional Dependency

This applies when you have a Composite Key (a Primary Key made of two or more columns). A dependency is "full" only if you need both parts of the key to find the result. If {Dish, Ingredient} is key. Then,

Example: In our Food Table,

$(\text{Dish} + \text{Ingredient}) \rightarrow \text{Amount}.$

You can't know the "Amount" just by knowing "Omelette." You also need to know the "Ingredient" (Egg) to know the amount is "2."

3. Transitive Dependency

Transitive Dependency occurs when a non-key attribute depends on another non-key attribute, rather than depending directly on the Primary Key.

It creates a "link in a chain" where:

- Column A (Primary Key) determines Column B.
- Column B determines Column C.
- Therefore, Column A determines Column C indirectly.

The Chain in this Example:

1. Dish → Difficulty Score:

The score is decided by which dish we are making
(e.g., Toast = 1)

2. Difficulty Score to Level Name:

The name "Easy" is decided by the score 1, not by the word "Toast."

Dish (PK)	Difficulty Score	Level Name
Toast	1	Easy
Omelette	4	Medium
Latte	7	Hard
Soufflé	9	Hard

4. Trivial Dependency

It is one where the information you are looking for is already contained within the information you started with.

Using your Food Table with the composite key (Dish + Ingredient):

- $(\text{Dish} + \text{Ingredient}) \rightarrow \text{Dish}$ is a trivial dependency
- $(\text{Dish} + \text{Ingredient}) \rightarrow \text{Ingredient}$ is a trivial dependency.

5. Partial Dependency

This happens when a column depends on only one part of a Composite Key rather than the whole thing.

Example: In a table with

(Dish + Ingredient)

as the key, adding a Difficulty column.

The Difficulty (Easy/Hard) depends only on the Dish. It doesn't care about the ingredient. This is a partial dependency because it "partially" ignores the key.

Therefore,
to change 1NF to 2NF, we should get rid of
partial dependencies.

Second Normal Form

In the First Normal Form of table Dishes, we know that it is in 1NF. But, it didn't violate the laws of 2NF either.

So, to violate the laws of 2NF, we add a column difficulty as ->

Dish	Ingredient	Amount	Unit	Difficulty
Omelette	Egg	2	pc	Easy
Omelette	Oil	1	tbsp	Easy
Sandwich	Bread	2	pc	Medium
Sandwich	Cheese	1	pc	Medium
Toast	Bread	2	pc	Easy
Latte	Espresso	1	tbsp	Hard
Latte	Steamed milk	0.5	cup	Hard

Second Normal Form...

Before adding Difficulty, table was in 2NF because every column needed the whole primary key (Dish + Ingredient).

Once you add Difficulty, you create a Partial Dependency i.e. Difficulty only needs the Dish to be determined.

So, we now have a table that is 1NF but NOT 2NF.

Dish	Ingredient	Amount	Unit	Difficulty
Omelette	Egg	2	pc	Easy
Omelette	Oil	1	tbsp	Easy
Sandwich	Bread	2	pc	Medium
Sandwich	Cheese	1	pc	Medium
Toast	Bread	2	pc	Easy
Latte	Espresso	1	tbsp	Hard
Latte	Steamed milk	0.5	cup	Hard

Second Normal Form

Dish	Ingredient	Amount	Unit	Difficulty
Omelette	Egg	2	pc	Easy
Omelette	Oil	1	tbsp	Easy
Sandwich	Bread	2	pc	Medium
Sandwich	Cheese	1	pc	Medium
Toast	Bread	2	pc	Easy
Latte	Espresso	1	tbsp	Hard
Latte	Steamed milk	0.5	cup	Hard

Dish	Ingredient	Amount	Unit
Omelette	Egg	2	pc
Omelette	Oil	1	tbsp
Sandwich	Bread	2	pc
Sandwich	Cheese	1	pc
Toast	Bread	2	pc
Latte	Espresso	1	tbsp
Latte	Steamed milk	0.5	cup

Dish	Difficulty
Omelette	Easy
Sandwich	Medium
Toast	Easy
Latte	Hard

For Third Normal Form

Dish	Ingredient	Amount	Unit
Omelette	Egg	2	pc
Omelette	Oil	1	tbsp
Sandwich	Bread	2	pc
Sandwich	Cheese	1	pc
Toast	Bread	2	pc
Latte	Espresso	1	tbsp
Latte	Steamed milk	0.5	cup

Dish	Difficulty
Omelette	Easy
Sandwich	Medium
Toast	Easy
Latte	Hard

Third Normal Form...

Before adding Difficulty Level, the table was in 2NF (and 3NF) because Difficulty depended directly and solely on the primary key (Dish). There was no "middle-man" attribute.

Once you add Difficulty Level (1–9): We create a Transitive Dependency. The Difficulty category (Easy/Medium/Hard) now actually depends on the Level number, which in turn depends on the Dish.

We now have a table that is still 2NF, but it is NOT 3NF because a non-key attribute (Difficulty) depends on another non-key attribute (Level).

Dish (PK)	Difficulty	Difficulty_Level
Omelette	Easy	1
Sandwich	Medium	5
Toast	Easy	2
Latte	Hard	8

Third Normal Form...

In your previous table, the following functional dependencies existed:

- Dish -> Difficulty
- Difficulty -> Difficulty_Level

Because Dish determines Level and Level determines Label, there is a Transitive Dependency (Dish -> Label).

To reach 3NF, we must eliminate this by splitting the table into two relations.

Dish (PK)	Difficulty_Level
Omelette	1
Sandwich	5
Toast	2
Latte	8

Thus, in 3NF, every non-key attribute must depend on the key, the whole key, and nothing but the key.

But, for composite key tables, rules for 3NF are:

In any dependency $X \rightarrow Y$, it is said to be in 3NF if

1. X is a super-key
2. Y is a prime attribute (member of the primary key)

However, the rules for BCNF are:

In any dependency $X \rightarrow Y$, it is said to be in BCNF if

1. X is a super-key
- ~~2. Y is a prime attribute (member of the primary key)~~

BUT, in BCNF, every attribute must depend on the key, the whole key, and nothing but the key.

Boyce-Codd Normal Form.....

It is stricter version of 3NF and is sometimes referred as 3.5NF. Now,

Imagine a table for University Tutoring.

Rules:

- A student can have many tutors (for different subjects).
- Each tutor specializes in only one subject.
- For every subject, a student is assigned to exactly one tutor.

Boyce-Codd Normal Form

Here,

Composite Primary Key is
{student, subject}

But, tutor (which is not PK, can
determine subject)

Which BCNF does not allow, and
to fix this, we must create a new
column where tutor is a PK.

Student (PK)	Subject (PK)	Tutor
Alice	Math	Prof. Gordon
Bob	Math	Prof. Smith
Alice	Physics	Prof. Hawking
Charlie	Physics	Prof. Feynman
Bob	Biology	Prof. Curie

Boyce-Codd Normal Form

Here,

Composite Primary Key is
{student, subject}

The dependencies are:

1. { Student, Subject } $\rightarrow$ Tutor
2. Tutor $\rightarrow$ Subject

Student (PK)	Subject (PK)	Tutor
Alice	Math	Prof. Gordon
Bob	Math	Prof. Smith
Alice	Physics	Prof. Hawking
Charlie	Physics	Prof. Feynman
Bob	Biology	Prof. Curie

Boyce-Codd Normal Form...

Subject (PK)	Tutor
Math	Prof. Gordon
Math	Prof. Smith
Physics	Prof. Hawking
Physics	Prof. Feynman
Biology	Prof. Curie

Student (PK)	Tutor
Alice	Prof. Gordon
Bob	Prof. Smith
Alice	Prof. Hawking
Charlie	Prof. Feynman
Bob	Prof. Curie

3NF vs BCNF

.. Assignment

ARMSTRONG AXIOMS

Axiom	Rule	Explanation
Reflexivity	If $Y \subseteq X$ then $X \rightarrow Y$	If you know (ID, Name), you automatically know ID (Trivial).
Augmentation	If $X \rightarrow Y$ then $XZ \rightarrow YZ$	If $ID \rightarrow Name$, then $(ID, Age) \rightarrow (Name, Age)$.
Transitivity	If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$	If $ID \rightarrow Dept$ and $Dept \rightarrow Dean$, then $ID \rightarrow Dean$.

Armstrong's Axioms are a set of fundamental rules (inference rules) used to derive all the functional dependencies that logically follow from a given set of initial dependencies.

Attribute Closure for Key Identification

Given a relation R with attributes {A, B, C, D, E} and the following Functional Dependencies (FDs):

- $A \rightarrow B$,
- $B \rightarrow C$,
- $C \rightarrow D, E$

Question: Find the closure of {A, D}, written as {A, D}⁺.